COMP 356 Sample Midterm Exam I

1. (8 points) What are the relative advantages disadvantages of implementing a programming language with an interpreter as compared with implementing the language with a compiler? Give at least two advantages for each approach.

2. Consider the following BNF definition:

   <S> → <A><A> | c<B><A>
   <A> → a<A> | <B> | ϵ
   <B> → c<B> | c

   (a) (5 points) Give a rightmost derivation for the string: ccaa

   (b) (10 points) Show that the BNF definition is ambiguous.

3. (20 points) Consider adding Java-style switch statements to the small language for which we gave denotational semantics in class. The BNF definition of statements is then expanded to:

<stmt> → ...
| switch (<expr>) {<cases>}
<cases> → case <expr>: <sl> break; <cases> | ϵ

The semantic function M_{stmt} for switch statements is:

M_{stmt}(switch (<expr>) {<cases>}, s) =
if $s = \perp$ then $\perp$
else let $v = M_{expr}$(<expr>, s) in
if $v = \perp$ then $\perp$
else M_{cases}(<cases>, v, s)

Give the definition of the semantic function M_{cases}. As in Java, the values of the expressions for each case are tried in order, and when the value of one of these expressions matches the value of the selector (the selector is the expression immediately following the keyword switch), the statements in that case are executed. Because the syntax of this language requires a break after each case, no statements in following cases are executed. If none of the expressions in the cases have the same value as the selector, then none of the cases are executed. Note that the value of the selector is passed as the second argument to M_{cases}, so that M_{cases} is a function that takes three arguments (<cases>, a value and a state) and returns $state_{\perp}$.

4. (6 points) Write an EBNF definition the describes the set of all strings `0`'s and `1`'s that contain at least three `1`'s.

5. (6 points) Give a regular expression that generates the set of all strings of `a`'s, `b`'s and `c`'s such that all `c`'s appear after any `a`.

6. (20 points) Consider the following grammar for boolean expressions, where expr is a nonterminal, and tokens LESS, GREATER, AND, OR, NOT, LPAR, RPAR and NUM have their usual meanings, i.e. LESS means < and so on, except that NUM represents only integers.

   expr → expr LESS expr | expr GREATER expr | expr AND expr | expr OR expr | NOT expr
   | LPAR expr RPAR | NUM

   Write a CUP specification for a program that takes an expression formed according to this grammar as input, and prints the value of the expression as output. Boolean values are represented as integers: 0 is false, and any other integer is true. The result of applying any relational or boolean operator listed above is 0 or 1. You can assume that a lexical analyzer for this grammar is available, so you do not need to worry about where the input is coming from. Additionally, you can assume that the lexical analyzer sets the attribute value for token NUM to be the `Integer` lexeme for the token. The Java method to return the `int` from an instance of class `Integer` is `intValue()`.

   You must declare the tokens listed above in your CUP specification, and you must ensure that the parser generated from your specification has no shift-reduce or reduce-reduce conflicts. The relational operators LESS and GREATER do not associate. All of the other binary operators are left associative, and NOT is right associative (at least for purposes of this problem). The precedence order from lowest to highest is: OR, then AND, then NOT and then the relational operators. That is, LESS and GREATER have equal (and highest) precedence.

   You do not need to provide the imports or directives sections of the CUP specification.

   Your CUP specification must print exactly one value as output. That is, your specification should print only the value of the entire input expression, and not the values of any subexpressions.

   Write your answer for this problem on the following page (which has intentionally been left blank).

This page intentionally left blank.

7. Consider the following declarations.

```
typedef struct foo {
  int x;
  double y;} footype;
typedef struct bar {
  int x;
  double y;} bartype;
typedef footype baztype;
typedef bartype bar2type;
typedef baztype zabtype;

footype a;
bartype b;
baztype c;
zabtype d;
bar2type f;
```

(a) (5 points) List all variables whose type is compatible with the type of `a` under structural type compatibility.

(b) (6 points) List all variables whose type is compatible with the type of `a` under the type compatibility rules of C++.

(c) (5 points) List all variables whose type is compatible with the type of `a` under name type compatibility.